

BiteBuddy

AI-Powered Food Delivery and Customer Support Platform

PORTFOLIO DOCUMENT PURPOSE

A professional Business Analysis and product-delivery record covering the business problem, scope, requirements, customer journeys, AI controls, architecture, API strategy, testing evidence, defect resolution, closure and future production roadmap.

Field	Detail
Document version	2.0
Document status	Project Closed - Approved for Closure
Document date	xx April 20xx
Implementation boundary	Mobile-first responsive React prototype using fictional operational data
Final repository checkpoint	fac9d05
Final automated test result	68 passed; 0 failed

CONFIDENTIALITY AND CLAIM BOUNDARY

This document contains fictional or mock business data only. It is portfolio evidence of analysis, design, implementation and testing practice; it is not evidence of live restaurant, courier, payment, banking or customer-support integration.

. Document Map

The document has been refreshed from the earlier Stage 3B checkpoint pack to reflect final QA, Product Owner closure approval, 68 passing automated tests and all completed delivery checkpoints.

Section	Title	Purpose
1	Executive Summary	Final product position, value and verified evidence
2	Business Context and Objectives	Problem, objectives, benefits and success measures
3	Vision, Scope and Principles	Product boundary, assumptions and operating principles
4	Stakeholders and Governance	Roles, RACI, decisions and closure authority
5	Delivery Approach and Checkpoints	Stages, Git checkpoints and change control
6	Delivered Capability	Food discovery, orders, support and AI functions
7	Customer Journeys	Mobile interaction and key end-to-end journeys
8	Solution Architecture	Frontend, backend, providers, rules and data authority
9	AI Controls and Safety	Guardrails, labels, validation and fallback
10	Functional Requirements	Traceable business and platform requirements
11	Non-Functional Requirements	Security, privacy, quality and resilience
12	Data and Integration Strategy	Prototype entities and future APIs
13	User Stories and Acceptance Criteria	Portfolio-ready Agile requirements evidence
14	Testing and Quality Evidence	QA catalogue, test results and defects
15	RAID and Controls	Risks, assumptions, issues and dependencies
16	UAT, Closure and Release Position	Acceptance, sign-off and production boundary
17	Business Analysis Evidence	BA contribution, artefacts and interview value
18	Roadmap and Recommendations	Production-readiness priorities
A-D	Appendices	RTM, test catalogue, defect log and glossary

Document control

Field	Detail
Product	BiteBuddy: AI-Powered Food Delivery and Customer Support Platform
Document type	Professional project delivery and Business Analysis documentation
Data classification	Portfolio-safe; mock data and simulated actions
Approval boundary	Prototype approved for closure by the Product Owner
Production boundary	Live integrations, production security certification and operational monitoring remain future work

1. Executive Summary

BiteBuddy is a mobile-first responsive food-delivery and customer-support prototype that combines natural-language meal discovery, preference and allergy handling, cart and mock checkout, order support and a controlled AI interpretation layer. The product was treated as a real delivery project with staged scope, requirements, testing, defect management, Git checkpoints and Product Owner approval.

The architecture deliberately separates language understanding from business authority. Gemini or the mock provider can identify intent and constraints, but only deterministic BiteBuddy rules can choose menu records, check order eligibility, update local mock state or present support outcomes.

FINAL VERIFIED POSITION

Project closed and approved for closure. Final automated regression: 68 passed, 0 failed. Production build passed. Browser QA passed for the agreed journeys. Working tree was clean after commit fac9d05.

Delivered business value

- **Lower customer effort** - Customers can ask for meals, tracking and support using natural language rather than navigating separate screens.
- **Safer recommendations** - Dietary preferences, budgets and locked allergy exclusions are visible and enforced outside the AI model.
- **Consistent self-service** - Tracking, cancellation, delivery instructions, delivery address, refund status and payment-failure guidance follow controlled flows.
- **Transparent AI** - The UI distinguishes Gemini-assisted interpretation, mock interpretation and deterministic fallback.
- **Resilience** - Quota or provider failure does not remove safe support; the product can revert to mock or rule-based operation.
- **Portfolio evidence** - The project demonstrates requirements engineering, API planning, AI governance, QA, defect resolution and release control.

Final delivery position

Delivery stage	Scope	Final status
Stage 1	Core mobile-first food discovery and support experience	Completed
Stage 2	Multi-turn preferences, cart, mock checkout, orders and guided support	Completed
Stage 3A	Secure provider-independent AI foundation and deterministic fallback	Completed
Stage 3B	Server-side Gemini interpretation, schema validation and source labels	Completed
Formal QA	Regression, browser journeys, defect resolution and responsive UX	Completed
Closure	Product Owner acceptance and clean Git checkpoint	Approved and closed

2. Business Context and Objectives

Business problem

Food-delivery customers often need help with meal discovery, dietary constraints, allergies, order status, cancellation, delivery changes, payment questions and post-order issues. Traditional menus and help centres split these needs across separate screens and repeat information, increasing effort and creating inconsistent support outcomes.

BiteBuddy addresses this through one conversational entry point, while retaining strict business, safety and data boundaries.

Business objectives

ID	Objective
BO-01	Provide natural-language food discovery using visible preferences and constraints.
BO-02	Prevent unsafe recommendations by enforcing allergy and dietary exclusions deterministically.
BO-03	Enable self-service for common order and support scenarios.
BO-04	Reduce dependency on one AI provider through mock and rule-based fallback.
BO-05	Keep AI use transparent and prevent unsupported claims or actions.
BO-06	Create a secure, testable architecture that can connect to live APIs later.
BO-07	Deliver a mobile-first experience with readable cards and touch-friendly controls.
BO-08	Capture delivery evidence suitable for professional BA and AI-product portfolios.

Expected benefits and future KPIs

Measure	Definition
Customer effort	Reduced navigation steps and repeated explanations.
Constraint compliance	No recommendation violates a locked allergy or dietary rule.
Self-service completion	Eligible support journeys complete without unnecessary escalation.
Escalation quality	Unsupported, ambiguous or sensitive issues are routed safely.
Fallback continuity	Safe assistance remains available during quota or provider failure.
Mobile usability	Core journeys remain readable with no hidden critical controls.
Transparency	Every response shows the correct interpretation source and mock-data boundary.
Release quality	No critical or high-severity unresolved defect at sign-off.

MEASUREMENT BOUNDARY

The prototype does not contain production usage or commercial performance data. These are target measures for a future live service, not claimed realised benefits.

3. Product Vision, Scope and Principles

Vision statement

VISION

Make food ordering and support feel like a safe conversation: the customer explains the need, BiteBuddy interprets it, trusted rules decide and the interface returns a clear next step.

Product positioning

BiteBuddy is a mobile-first responsive web application with an advanced AI-assisted chatbot. It is not a native iOS or Android application. The implemented prototype runs locally and uses mock restaurants, menu items, customer data, orders, activities and support tickets.

In scope - delivered prototype

- Responsive web interface for mobile and desktop browsers.
- Natural-language meal discovery with multi-turn context and visible filter chips.
- Dietary preferences, budget interpretation and locked allergy exclusions.
- Grounded menu recommendations, closest alternatives, customisation and persistent cart.
- Mock checkout, latest-order selection, tracking and order item visibility.
- Guided support for late, missing, wrong, cold and damaged food issues.
- Controlled cancellation, delivery-instruction and delivery-address changes.
- Read-only refund-status and payment-failure guidance using recorded mock data only.
- Human-support escalation and simulated ticket/queue behaviour.
- Server-side Gemini integration, mock provider, schema validation and safe fallback.
- Responsive chatbot improvements, including a larger response area and horizontally scrollable quick replies.

Out of scope - current prototype

- Live restaurants, stock, catalogue, prices, availability or allergen ownership.
- Production authentication, identity proofing, consent records and account recovery.
- Live order management, courier location or delivery operations.
- Payment processing, bank/card diagnostics, refunds, compensation or settlement.
- Live support-agent platform, workforce management or SLA enforcement.
- Production email, SMS or push notifications.
- Native app packaging, app-store release, hosting, monitoring and regulatory certification.
- Product returns after food delivery; this was assessed as not applicable to the food-delivery business model.

Design principles

ID	Principle	Application
P1	AI interprets; rules decide	Language becomes structured intent. Deterministic logic controls outcomes.
P2	Safety over convenience	Allergy locks, confirmation and escalation take priority.
P3	Truthful experience	Source labels and mock-data disclosures must be accurate.
P4	Graceful degradation	Provider or quota failure must not remove safe assistance.
P5	Data minimisation	Use fictional data and keep secrets outside client code and Git.
P6	Mobile clarity	Use readable cards, visible constraints and touch-friendly controls.
P7	Testability	Provider boundaries, schemas and state changes must be independently testable.

4. Stakeholders and Delivery Governance

Stakeholder groups

Stakeholder	Role in product	Primary interest
Customer	Uses food discovery, ordering and support journeys.	Safe, clear and low-effort completion.
Product Owner	Owns value, priority, scope and acceptance.	Quality, business fit and closure confidence.
AI Business Analyst	Defines requirements, journeys, rules, risks and traceability.	Clear, testable outcomes and portfolio evidence.
Developer / Codex	Implements and inspects code and automated tests.	Secure and maintainable delivery.
QA Tester	Executes functional, regression and responsive checks.	Evidence-based quality and defect control.
DevOps Engineer	Manages local configuration, build and Git checkpoints.	Secret protection and reproducibility.
Support Operations	Owns future support policy and escalation handling.	Consistent classification and hand-off.
Security and Privacy	Reviews secrets, data handling and AI risk.	Least privilege, privacy and auditability.
Integration Owners	Own future order, courier, payment and support APIs.	Reliable contracts and authoritative data.

RACI summary

Activity	PO	AI BA	Developer	QA	DevOps
Product scope and priority	A	R	C	C	C
Requirements and acceptance criteria	A	R	C	C	C
Code and automated tests	C	C	R/A	C	C
Functional and regression testing	A	C	C	R	C
Git checkpoint and secret controls	C	C	R	C	A/R
UAT and closure approval	A/R	C	C	C	C
AI guardrails and risk controls	A	R	C	C	C

Governance and decision rules

- A change is not marked complete until the relevant automated, build and browser checks pass.
- State-changing support actions require deterministic eligibility and explicit customer confirmation.
- A defect is closed only after a focused retest and a clean Git checkpoint.
- AI provider output is advisory interpretation, never direct operational authority.
- Product Owner approval closes the prototype scope; future live integrations require a new release cycle.

5. Delivery Approach and Repository Checkpoints

Delivery used staged increments, one-task-at-a-time control, explicit Jira status, targeted defect investigation, automated regression, browser retesting and focused Git commits. This protected working functionality while new support journeys were introduced.

Delivery stages

Stage	Focus	Status
Stage 1	Core UI, menu, cart and basic support	Complete
Stage 2	Multi-turn context, preferences, mock orders and expanded support	Complete
Stage 3A	Provider abstraction, mock provider and rule fallback	Complete
Stage 3B	Secure Gemini interpretation and structured contract	Complete
Formal QA	End-to-end browser testing and defect correction	Complete
Portfolio readiness	Documentation and evidence capture	Current documentation output

Key repository checkpoints

Commit	Purpose
feb0303f	Complete BiteBuddy Stage 3A secure AI foundation
a668b8b	Complete BiteBuddy Stage 3B Gemini integration
3ea6e39	Fix Brick Lane Curry Club image
b450b1f	Fix BiteBuddy multi-turn preference retention
5e6e38f	Fix latest order tracking and confirmation popup
0b52def	Fix latest order cancellation routing
a0f89d2	Fix latest order delivery instruction flow
24147c3	Add latest order delivery address flow
3de998a	Add latest order refund status flow
4ee6124	Increase chatbot response area
fac9d05	Add latest order payment failure flow

CHANGE CONTROL EVIDENCE

Each checkpoint was preceded by targeted tests, production build verification or browser retesting as appropriate. The final working tree was clean after the latest commit.

6. Delivered Capability

Food discovery and customer preference capability

Capability	Delivered behaviour
Conversational search	Natural-language meal request.
Multi-turn context	Preferences and constraints retained across messages.
Visible filter chips	Active constraints remain understandable.
Dietary handling	Supported diet preferences are applied.
Allergy protection	Declared exclusions are locked and enforced.
Budget interpretation	Customer budget becomes a structured constraint.
Grounded results	Only authoritative mock menu records are displayed.
Closest alternatives	Nearest safe option explained when no exact match exists.
Customisation and cart	Items can be selected, customised and retained.

Order and support capability

Capability	Delivered behaviour
Mock checkout	Address, payment method, tip and cart total captured for simulation.
Latest order tracking	Newest order selected using createdAt; items and quantities displayed.
Cancellation	Eligibility and explicit confirmation protect state changes.
Delivery instructions	Proposed instruction displayed and applied only after confirmation.
Delivery address	Newest eligible order selected; address changes require confirmation.
Refund status	Read-only response; no invented approval, amount or date.
Payment failure	Read-only safe guidance; no invented bank or provider cause.
Order issues	Late, missing, wrong, cold and damaged food support routes.
Human support	Simulated ticket creation and support queue.
Product return	Not applicable after food delivery; removed from scope.

Platform and AI capability

Capability	Delivered behaviour
Secure frontend-to-backend pattern	Browser calls the BiteBuddy API; Gemini key stays server-side.
Provider abstraction	Gemini, mock and rule fallback selected by server configuration.
Structured interpretation	Provider output follows a strict shared contract.
Validation and error mapping	Invalid or unavailable output fails safely.
Deterministic authority	Business rules remain the final decision layer.
Transparent labels	UI shows AI-assisted, Mock assistant or BiteBuddy assistant.
Quota resilience	RPD/provider limits can be handled by mock or fallback mode.
Responsive chat area	Larger reading area and one-row quick replies improve usability.

7. Customer Journeys and Process Design

Customer journey: from need to safe outcome

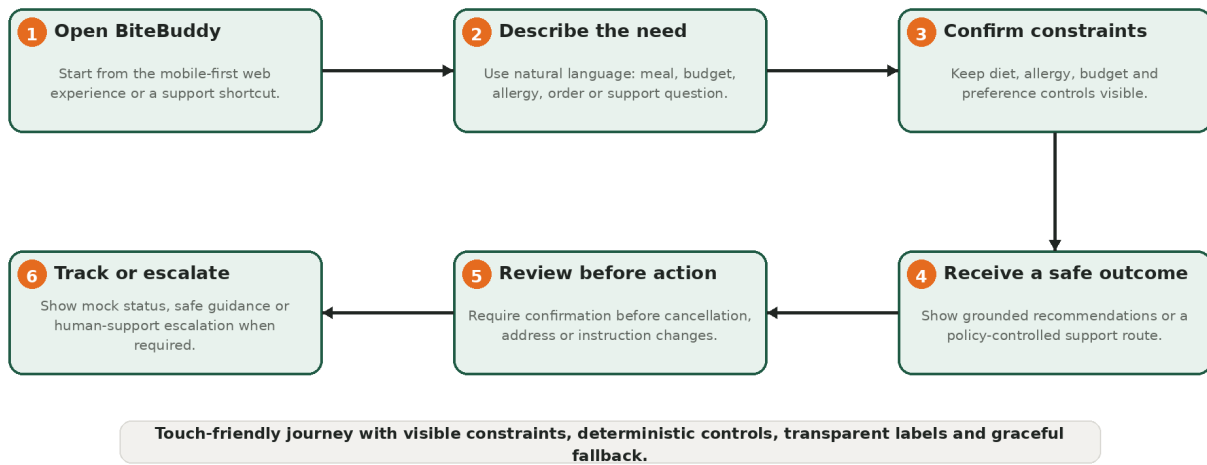


Figure 1. Mobile-first customer journey from request to grounded outcome, confirmation or escalation.

Key customer journeys

ID	Journey	Summary
CJ-01	Food discovery	Customer describes cuisine, diet, allergy, budget or mood; BiteBuddy returns grounded safe menu records.
CJ-02	Multi-turn refinement	Customer adds or changes constraints without losing locked safety preferences.
CJ-03	Cart and mock checkout	Customer selects an item, changes quantity/tip, enters mock delivery details and places a simulated order.
CJ-04	Track latest order	Newest order and item quantities are displayed with a mock-data boundary.
CJ-05	Cancel latest order	Eligibility is checked; explicit confirmation is required; refund is not promised.
CJ-06	Change delivery instructions	Existing and proposed instructions are shown before confirmation.
CJ-07	Change delivery address	Existing and proposed address are shown before a local mock update.
CJ-08	Refund status	Recorded data only; no fabricated date, amount or approval.
CJ-09	Payment failure	Recorded data only; safe security guidance and no bank/provider claim.
CJ-10	Report order issue	Guided late, missing, wrong, cold or damaged-food support route.
CJ-11	Human support	Simulated ticket and queue position created when escalation is appropriate.

8. Solution Architecture

BiteBuddy solution architecture and authority boundary

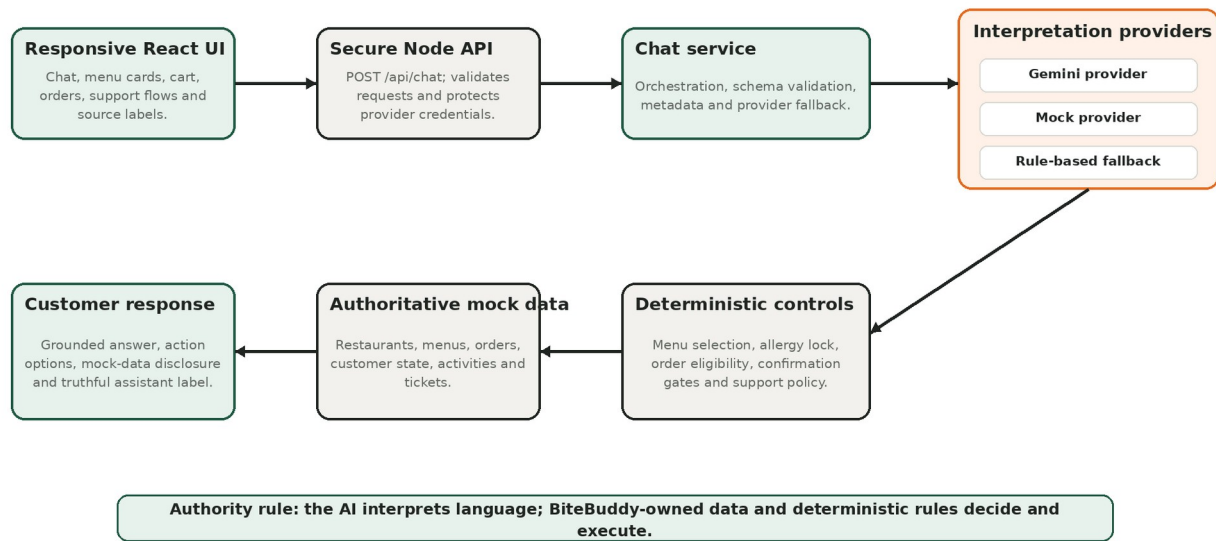


Figure 2. Solution architecture and authoritative decision boundary.

Architecture components

Component	Responsibility
Responsive React interface	Presents chat, recommendations, orders, support actions and labels.
Vite development/proxy layer	Hosts the local frontend and routes API requests.
Node chat API	Accepts POST /api/chat and validates the shared request/response contract.
Chat service	Coordinates provider interpretation, validation, metadata and fallback.
Gemini provider	Uses the official server-side SDK for language interpretation.
Mock provider	Supports deterministic development and testing without a live request.
Rule fallback	Maintains safe core assistance during provider failure.
Deterministic handlers	Select records, apply policy, enforce eligibility and control state changes.
Mock state/data	Represents restaurants, menu, customers, orders, activities and tickets.

Architecture decisions

ID	Decision	Rationale
ADR-01	Server-side AI access	Protects credentials and centralises controls.
ADR-02	Provider-independent contract	Allows Gemini, mock or future providers without rewriting journeys.
ADR-03	Structured output	Reduces ambiguity and makes validation testable.
ADR-04	Deterministic business authority	Prevents model output becoming an unauthorised action.
ADR-05	Rule-based fallback	Preserves continuity during quota or provider failure.
ADR-06	Mock operational data	Enables safe prototyping without personal or transaction data.
ADR-07	Transparent source labels	Prevents misleading claims about how the response was produced.

9. AI Controls, Guardrails and Transparency

Controlled AI interpretation flow

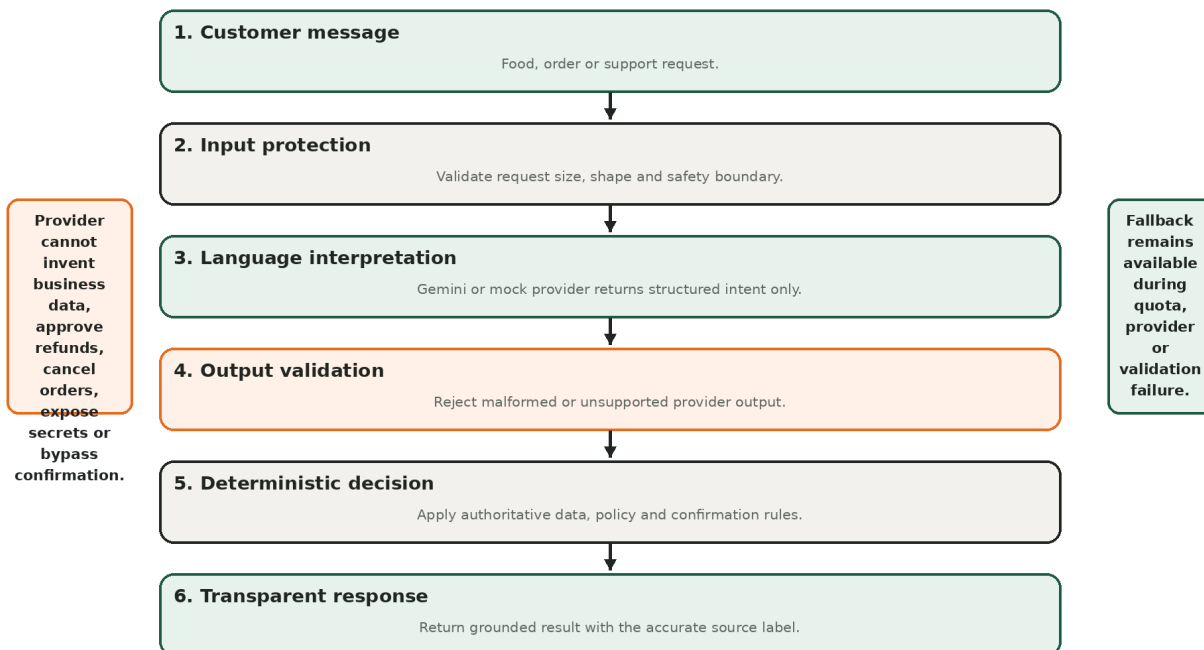


Figure 3. Controlled AI interpretation with schema validation, deterministic action and fallback.

AI role

Gemini is used to interpret customer language into a constrained structure, including intent, dietary preference, allergy, cuisine, budget and support category. It is not the source of truth for menu, order, courier, payment, refund or support records.

Prohibited AI actions

- Invent restaurants, menu items, prices, allergens, orders, payments or courier data.
- Choose a final business record without deterministic verification.
- Cancel an order or update address/instructions directly.
- Approve a refund, compensation or financial outcome.
- Claim access to a bank, card network, restaurant, courier or live support system.
- Bypass explicit customer confirmation.
- Reveal secrets, provider credentials, internal prompts or configuration.

Assistant source labels

Customer-visible label	Meaning
AI-assisted using mock business data	Gemini interpreted the message; final output still uses mock records and deterministic rules.
Mock assistant	The local mock provider interpreted the message.
BiteBuddy assistant	Deterministic fallback handled the message without a live provider result.

Prompt and output controls

Control	Purpose
Input validation	Reject malformed or oversized requests.
Instruction boundary	Provider is explicitly restricted to interpretation.
Structured contract	Allowed fields and values are shared across providers.
Schema validation	Invalid provider output is rejected safely.

Control	Purpose
Grounding	Final records come from BiteBuddy-owned data.
Confirmation gates	Sensitive local state changes require deterministic confirmation.
Safe error mapping	Provider failures become non-sensitive customer-safe behaviour.
Quota handling	RPD or provider limits trigger mock/fallback operation rather than unsafe retries.

10. Functional Requirements

ID	Area	Requirement	Status
FR-001	Chat	Accept food, order or support requests in natural language.	Implemented
FR-002	Context	Retain relevant preferences across the active conversation.	Implemented
FR-003	Context	Display active filters and locked exclusions.	Implemented
FR-004	Food discovery	Apply supported dietary preferences.	Implemented
FR-005	Safety	Lock and enforce declared allergy exclusions.	Implemented
FR-006	Food discovery	Interpret and apply a customer budget.	Implemented
FR-007	Grounding	Return only menu items from authoritative mock data.	Implemented
FR-008	Alternatives	Explain the closest safe option when no exact match exists.	Implemented
FR-009	Cart	Customise items and retain cart state.	Implemented
FR-010	Checkout	Capture mock address, payment method, tip and place a simulated order.	Implemented
FR-011	Orders	Select the latest order using reliable createdAt ordering.	Implemented
FR-012	Tracking	Display order reference, restaurant, items, quantities and mock status.	Implemented
FR-013	Cancellation	Evaluate eligibility before cancellation.	Implemented
FR-014	Confirmation	Require explicit confirmation before simulated state changes.	Implemented
FR-015	Delivery	Allow controlled delivery-instruction changes.	Implemented
FR-016	Delivery	Allow controlled delivery-address changes.	Implemented
FR-017	Refund	Provide read-only refund-status information from recorded mock data only.	Implemented
FR-018	Payment	Provide read-only payment-failure guidance without inventing a cause.	Implemented
FR-019	Issues	Support late, missing, wrong, cold and damaged-food categories.	Implemented
FR-020	Support	Allow escalation and create a simulated support ticket.	Implemented
FR-021	AI platform	Support configurable Gemini, mock and fallback modes.	Implemented
FR-022	Security	Keep provider access and credentials server-side.	Implemented
FR-023	Validation	Validate provider output against the shared contract.	Implemented
FR-024	Fallback	Use deterministic fallback when the provider is unavailable.	Implemented
FR-025	Transparency	Display the correct assistant source label.	Implemented
FR-026	Safety	Prevent provider output from directly executing a business action.	Implemented
FR-027	Errors	Return controlled behaviour for quota and provider errors.	Implemented
FR-028	Responsive UI	Support readable mobile-first chat, cards and support actions.	Implemented and browser tested
FR-029	Response area	Provide a larger scrollable response area and accessible quick replies.	Implemented and browser tested
FR-030	Scope rule	Treat post-delivery product return as not applicable for prepared food.	Approved scope decision

11. Non-Functional Requirements

ID	Category	Requirement	Status
NFR-001	Security	Credentials never appear in client code, screenshots, logs or Git.	Met for prototype
NFR-002	Privacy	Prototype testing uses fictional data only.	Met
NFR-003	Reliability	Provider failure degrades to safe assistance.	Met
NFR-004	Truthfulness	UI accurately identifies response source and mock boundary.	Met
NFR-005	Maintainability	Provider logic is isolated behind a shared interface.	Met
NFR-006	Testability	Provider success, errors, fallback and state changes are testable.	Met
NFR-007	Performance	UI avoids indefinite waiting and keeps local flows responsive.	Met for prototype
NFR-008	Responsive design	Core journeys remain usable on mobile and desktop.	Browser validated
NFR-009	Accessibility	Keyboard focus visible; full production audit still required.	Partially met
NFR-010	Observability	Safe error codes without secrets or message content.	Production design required
NFR-011	Scalability	Rate limits, RPD and traffic spikes need capacity controls.	Production design required
NFR-012	Resilience	Provider substitution does not rewrite customer journeys.	Met
NFR-013	Data integrity	AI output cannot overwrite authoritative data directly.	Met
NFR-014	Compliance	Privacy, consumer, accessibility and AI-governance review.	Required before production
NFR-015	Release quality	No critical/high unresolved defect at closure.	Met for prototype

Security and privacy controls evidenced

- Real Gemini API key stored only in an ignored local .env file.
- No API key shown in screenshots, copied into chat or committed to Git.
- Browser communicates with the local backend rather than Gemini directly.
- Payment guidance explicitly warns against sharing full card number, PIN, CVV, password or one-time code.
- Mock data and local updates are clearly disclosed to the customer.

12. Data Model and Integration Strategy

Prototype data entities

Entity	Prototype purpose
Customer profile	Mock identity, preferences, diet and allergy exclusions.
Restaurant	Mock restaurant identity and service attributes.
Menu item	Name, description, price, dietary tags, allergens and customisation.
Cart	Items, quantity, customisation, fees, tip and mock total.
Order	Reference, createdAt, status, items, address, instructions and activity history.
Order item	Item name, quantity and selected options.
Courier status	Simulated delivery state and tracking narrative.
Support ticket	Simulated category, order reference and queue state.
Conversation context	Intent, filters, locked exclusions and confirmation state.
Provider metadata	Interpretation source and safe error status.

AUTHORITATIVE DATA RULE

The AI provider may interpret what the customer means, but it never becomes the source of truth. All records and permitted actions must be validated against BiteBuddy-owned data and policy.

Current API boundary

Interface	Responsibility
POST /api/chat	Receives customer message and context; returns structured interpretation metadata and safe response data.
Environment configuration	Selects gemini, mock or fallback provider on the server.
Shared AI contract	Defines allowed intents, constraints and validation rules.

Future production integration ecosystem

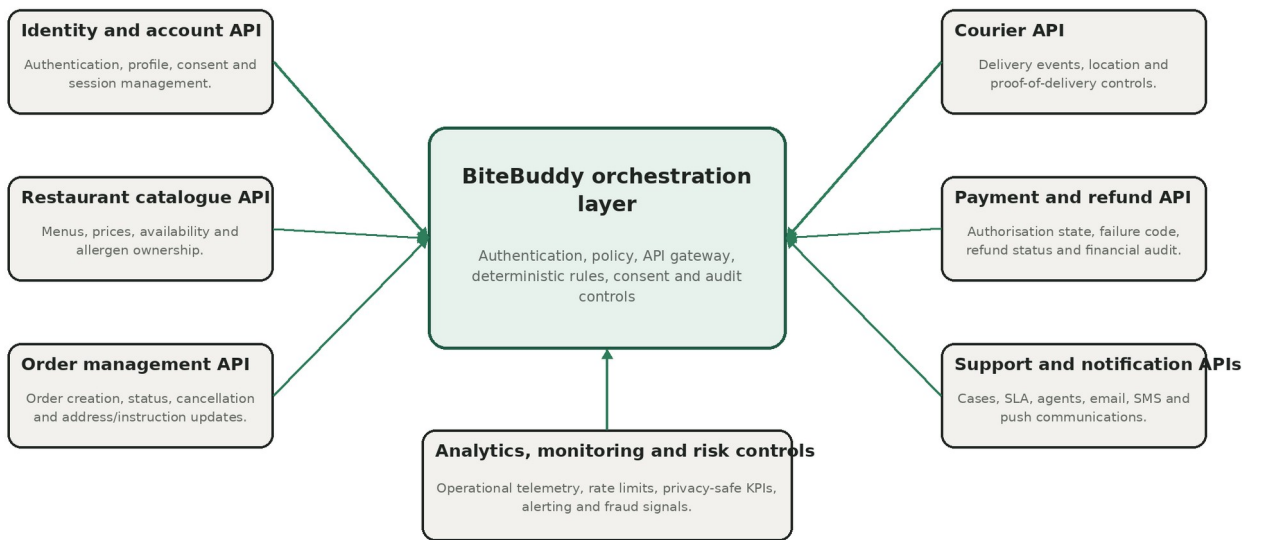


Figure 4. Future production API ecosystem; sensitive data remains behind controlled service boundaries.

13. User Stories and Acceptance Criteria

ID	User story	Acceptance criteria
US-01	As a customer, I want to describe dietary needs and budget in natural language so that I can find a suitable meal.	Given supported constraints, when I submit a request, then only grounded matching items or a safe alternative are shown.
US-02	As a customer with an allergy, I want the exclusion retained so that unsafe items never appear in later results.	Given a locked allergy, when I refine the request, then conflicting items remain excluded.
US-03	As a customer, I want to place a mock order so that I can demonstrate the complete journey.	Given a valid cart and mock details, when checkout is confirmed, then a new order with createdAt and items is stored locally.
US-04	As a customer, I want to track my latest order so that I see the newest mock status and ordered items.	Given multiple orders, when I ask for the latest, then the newest createdAt order is selected.
US-05	As a customer, I want to cancel an eligible order only after confirmation so that accidental cancellation is prevented.	Given an eligible order, when I request cancellation, then no state change occurs until explicit confirmation.
US-06	As a customer, I want to change delivery instructions safely so that I can review the proposed wording before it is saved.	Given a latest eligible order, when I propose instructions, then current and proposed values are shown and saved only after confirmation.
US-07	As a customer, I want to change my delivery address before dispatch so that the newest eligible order is updated safely.	Given a proposed address, when I confirm, then only the selected mock order address changes locally.
US-08	As a customer, I want honest refund information so that I am not given a fabricated date or approval.	Given no refund record, when I ask for status, then the response states no status/date is recorded and no guarantee is made.
US-09	As a customer, I want safe payment-failure guidance so that I know the next steps without exposing sensitive credentials.	Given no payment failure record, when I ask why payment failed, then no cause is invented and safe bank/provider guidance is shown.
US-10	As a Product Owner, I want AI actions constrained by rules so that the model cannot alter business state directly.	Given any provider output, when it is processed, then schema validation and deterministic rules control the final outcome.

Definition of Done used during QA

- Requirement and expected behaviour understood.
- Focused code change implemented without unrelated scope.
- git diff --check passes; only harmless Windows line-ending warnings accepted.
- Automated regression passes.
- Production build passes.
- Browser journey passes, including pre-confirmation and post-confirmation where relevant.
- Approved files staged, cached diff validated and focused commit created.
- Working tree confirmed clean.

14. Testing and Quality Evidence

Final technical evidence

Evidence item	Recorded result
Branch	main
Final checkpoint	fac9d05
Automated AI tests	68 passed; 0 failed
Production build	Passed; 1,588 modules transformed
Git diff/whitespace validation	Passed
Working tree after final commit	Clean
Secret protection	.env ignored; credentials not committed
Provider mode after screenshot attempt	Returned to mock mode after RPD limit
Product Owner decision	Approved for closure

Formal QA catalogue

Test ID	Scenario	Status
BB-QA-001	Application load and images	Passed
BB-QA-002	Vegetarian meal under £15	Passed
BB-QA-003	Multi-turn preference retention	Passed after correction
BB-QA-004	Explicit budget change	Passed
BB-QA-005	Locked peanut allergy cannot be overridden	Passed
BB-QA-006	Closest safe alternatives	Passed
BB-QA-007	Item selection	Passed
BB-QA-008	Add item to cart	Passed
BB-QA-009	Quantity increase and total	Passed
BB-QA-010	Tip changed to zero	Passed
BB-QA-011	Simulated checkout	Passed
BB-QA-012	Latest order tracking and items	Passed after correction
BB-QA-013	Latest order cancellation	Passed after correction
BB-QA-014	Latest order delivery-instruction change	Passed
BB-QA-015	Latest order delivery-address change	Passed
BB-QA-016	Latest order refund-status response	Passed
BB-QA-017	Latest order payment-failure response	Passed
BB-QA-018	Product return	Not applicable; removed from scope

AI provider evidence

- Automated Gemini adapter, schema, exact-phrase, label and error-mapping tests passed.
- A controlled live Gemini smoke test and browser test passed earlier using gemini-3.5-flash with the label "AI-assisted using mock business data".
- A later portfolio screenshot attempt encountered an RPD limit; the application was returned to mock mode without changing business data.
- Quota exhaustion was treated as a provider limitation, not as permission to fabricate a live response.

15. Defect and Change Summary

ID	Defect/change	Status	Resolution evidence
BB-UX-004	Missing Brick Lane Curry Club image	Resolved	Image corrected and committed.
BB-FUNC-017	Vegetarian preference lost on second turn	Resolved	Multi-turn retention corrected.
BB-FUNC-018	Budget reset incorrectly	Resolved	Explicit budget retained and updated correctly.
BB-TECH-005	Mock response rejected and fallback label shown	Resolved	Contract and label behaviour corrected.
BB-FUNC-019	Tracking UI omitted items	Resolved	Items and quantities added.
BB-FUNC-020	Latest checkout not selected for tracking	Resolved	createdAt ordering and local persistence corrected.
BB-UX-005	Order confirmation popup stayed open	Resolved	Auto-close after approximately four seconds.
BB-FUNC-021	"latest" matched "late"	Resolved	Intent matching corrected.
BB-FUNC-022	Cancellation ignored latest-order selection	Resolved	Newest eligible order routing corrected.
BB-FUNC-023	Delivery-instruction latest-order routing	Resolved	Extraction, confirmation and punctuation fixed.
BB-UX-006	Double punctuation in instruction messages	Resolved	Display-only formatting helper added.
BB-FUNC-024	Delivery-address request routed to food	Resolved	New support intent and safe confirmation flow.
BB-FUNC-025	Refund-status request routed to food	Resolved	Read-only recorded-data-only flow.
BB-UX-007	Chat response area too small	Resolved	Desktop height increased; quick replies scroll horizontally.
BB-FUNC-026	Payment-failure request routed to food	Resolved	Read-only safe guidance flow.
BB-TECH-API	Gemini RPD quota exceeded during screenshot capture	Controlled	Stopped retries and restored mock mode.

Quality lessons

- Intent keywords must be tested against similar words such as latest and late.
- Latest-order logic requires explicit timestamps and deterministic sorting.
- Display punctuation must be separated from stored customer text.
- Read-only support journeys must never invent operational or financial data.
- Provider quotas are an expected dependency and require clear fallback behaviour.
- UI improvements need visual confirmation in addition to automated tests.

16. RAID Log and Controls

ID	Risk	Rating	Control
R-01	Provider quota or availability blocks live interpretation.	Medium	Mock and rule fallback; controlled request frequency; future capacity plan.
R-02	AI returns malformed or unsupported structure.	Low	Strict schema validation and safe rejection.
R-03	AI invents menu, order, payment or refund data.	High	Interpretation-only role and deterministic grounding.
R-04	Allergy exclusion is lost in a follow-up.	High	Persistent lock and regression coverage.
R-05	Customer believes mock tracking or support is live.	Medium	Clear simulation disclosure and truthful labels.
R-06	API key is exposed.	High	Server-side provider access, ignored .env and Git checks.
R-07	Stale confirmation changes the wrong order.	High	State machine, latest-order selection and confirmation guards.
R-08	Responsive layout hides critical controls.	Medium	Flexible message area, quick-reply scrolling and browser QA.
R-09	Future APIs bypass business rules.	High	Contract-first integration and policy ownership.
R-10	Payment guidance exposes or requests sensitive data.	High	Security warning and no collection of card credentials.
R-11	Prototype claims are mistaken for live capability.	High	Portfolio-safe wording and explicit mock boundary.

Assumptions

- Mock menu and order records are internally consistent for prototype testing.
- Customer identity is not verified because authentication is outside scope.
- Local mock actions demonstrate workflow only and do not contact restaurants or couriers.
- Provider interpretation can be replaced without changing deterministic business rules.

Dependencies

- Stable local Node, npm and Vite environment.
- Gemini quota and API availability for live interpretation evidence.
- Future contracts from identity, restaurant, order, courier, payment, refund and support services.
- Product Owner policy decisions for cancellation, refund, escalation and customer communications.
- Security, privacy, accessibility and compliance review before production release.

17. UAT, Closure and Release Position

Minimum UAT acceptance criteria

ID	Acceptance criterion	Status
UAT-01	Natural-language food request returns grounded results.	Passed
UAT-02	Diet, budget and allergy context remains visible and correctly applied.	Passed
UAT-03	Locked allergy prevents conflicting recommendations.	Passed
UAT-04	Item selection, quantity, tip, cart and mock checkout work.	Passed
UAT-05	Latest mock order tracking displays order items.	Passed
UAT-06	Cancellation requires eligibility and confirmation.	Passed
UAT-07	Delivery instruction and address changes require confirmation.	Passed
UAT-08	Refund and payment responses do not fabricate financial facts.	Passed
UAT-09	Supported order issues follow controlled journeys.	Passed
UAT-10	Gemini, mock and fallback labels are truthful.	Passed in automated and controlled live evidence
UAT-11	Provider or quota failure does not break safe assistance.	Passed
UAT-12	No API key or secret appears in browser or repository evidence.	Passed
UAT-13	Chat response area, header and composer remain visible.	Passed
UAT-14	No critical/high unresolved defect remains.	Passed
UAT-15	Product Owner accepts the mock-data boundary and closure wording.	Approved

Closure statement

PROJECT CLOSED - APPROVED FOR CLOSURE

The Product Owner approved the agreed prototype scope after functional QA, automated regression, production build verification, browser retesting and clean Git checkpoints. Product returns were formally removed as not applicable to the food-delivery model.

Production release boundary

Closure applies to the portfolio prototype, not to a live commercial release. Production requires authenticated users, authoritative APIs, financial and support-system integration, security hardening, privacy and accessibility assurance, observability, capacity controls and operational ownership.

18. Business Analysis Evidence

BA contribution

BA area	Evidence in BiteBuddy
Problem framing	Defined customer effort, fragmented support and safety problems.
Scope management	Separated mobile-first prototype capability from future production integrations.
Stakeholder analysis	Mapped customer, Product Owner, delivery, QA, support, security and API owners.
Requirements engineering	Defined functional, non-functional, safety and transparency requirements.
Customer journeys	Structured discovery, cart, tracking, cancellation, delivery changes, refund and payment support.
AI guardrails	Specified interpretation-only AI use and deterministic authority.
Business rules	Defined latest-order selection, eligibility, confirmation and read-only financial guidance.
Acceptance criteria	Created UAT-ready conditions and Definition of Done.
Traceability	Linked objectives, requirements, tests, defects and closure status.
Risk management	Captured quota, allergy, secret, financial-data, layout and integration risks.
Agile governance	Used Jira statuses, focused defects, staged delivery and clean checkpoints.
Release governance	Distinguished prototype closure from production readiness.

Portfolio artefacts demonstrated

- Business problem, objectives and proposed success measures.
- Vision, scope, out-of-scope and design principles.
- Stakeholder map and RACI.
- Customer journeys and process diagrams.
- Functional and non-functional requirements.
- User stories and acceptance criteria.
- Solution architecture, AI control flow and API roadmap.
- Test catalogue, UAT criteria, defect log and traceability matrix.
- RAID log, closure decision and future roadmap.

Professional interview narrative

The project demonstrates how an AI Business Analyst can translate a customer-service problem into controlled product requirements, define AI guardrails, collaborate across development and QA, use evidence to close defects, and preserve a clear boundary between a safe prototype and a production service.

19. Roadmap and Recommendations

Priority	Workstream	Recommendation
1	Identity and consent	Add secure authentication, consent capture and customer-session controls.
2	Authoritative catalogue	Integrate restaurants, menu availability, price and allergen ownership.
3	Order and courier services	Connect real order state, delivery events and address/instruction changes.
4	Payment and refund services	Use provider-owned status codes, audit trails and safe customer messaging.
5	Support platform	Create real cases, agent assignment, SLA and customer communication.
6	Security and privacy	Threat model, secrets management, retention policy, DPIA and penetration testing.
7	Accessibility	Complete WCAG-focused keyboard, screen-reader, contrast and zoom audit.
8	Observability and scale	Rate limiting, safe logs, monitoring, capacity planning and alerting.
9	Evaluation and analytics	Measure constraint compliance, self-service completion and fallback rates.
10	Deployment	CI/CD, production hosting, environment separation and rollback controls.

Recommended production operating model

- Gemini interprets language but never receives unrestricted operational authority.
- Backend services retrieve only the minimum necessary data from live APIs.
- Deterministic policy validates every response and state change.
- Sensitive financial or personal data remains within controlled services.
- Human escalation is available for ambiguity, safety, payment disputes and policy exceptions.
- Provider quotas and outages are managed through rate limits, caching where appropriate and fallback.

NEXT PORTFOLIO ACTION

Publish this PDF with selected anonymised screenshots showing the customer interface, AI-assisted label, menu recommendations, order support and architecture. Do not show the API key, .env content, terminal IP details or personal information.

Appendix A. Requirements Traceability Matrix

Objective	Requirements	Evidence	Status
BO-01	FR-001 to FR-008	BB-QA-002 to BB-QA-006	Implemented and passed
BO-02	FR-004, FR-005, FR-023, FR-026	Allergy lock and deterministic tests	Passed
BO-03	FR-011 to FR-020	BB-QA-012 to BB-QA-017	Passed
BO-04	FR-021, FR-024, FR-027	Mock, Gemini and fallback tests	Passed
BO-05	FR-022, FR-025, FR-026	Schema, label and secret checks	Passed
BO-06	NFR-001, NFR-005, NFR-006, NFR-012	Build, Git and provider evidence	Passed for prototype
BO-07	FR-028, FR-029, NFR-008	Visual response-area test	Passed
BO-08	All documentation sections	Closure pack and portfolio PDF	Completed

Appendix B - Extended test catalogue

ID	Area	Scenario	Expected result
TC-001	Food discovery	Vegetarian meal under budget	Grounded results and visible constraints
TC-002	Conversation context	Add spicy requirement on follow-up	Diet and budget retained
TC-003	Allergy safety	Attempt to override peanut allergy	Unsafe items remain excluded
TC-004	Closest alternative	No exact match under tight constraints	Nearest safe option explained
TC-005	Cart	Select, add, increase quantity and remove tip	Totals update accurately
TC-006	Checkout	Place mock order	New order and createdAt stored
TC-007	Tracking	Ask for latest order	Newest order, items and status shown
TC-008	Cancellation	Cancel latest order	Eligibility and confirmation enforced
TC-009	Delivery instructions	Change latest order instruction	No mutation before confirmation
TC-010	Delivery address	Change latest address	Current and proposed address shown
TC-011	Refund status	Ask for latest refund date	No fabricated date or approval
TC-012	Payment failure	Ask why latest payment failed	No invented cause; safe guidance
TC-013	Provider labels	Use Gemini, mock and fallback	Correct label displayed
TC-014	Provider quota	RPD limit encountered	Retry stopped; mock mode restored
TC-015	Responsive UI	Long support reply	Larger response area; header/composer visible
TC-016	Security	Review .env and Git boundary	No secret exposure

Appendix C. Final Defect Status and Checkpoints

All formally recorded defects used for closure were resolved or controlled. No critical or high-severity unresolved defect remained at Product Owner approval.

Checkpoint	Outcome
a668b8b	Secure Gemini integration checkpoint.
b450b1f	Multi-turn preference retention corrected.
5e6e38f	Latest-order tracking and popup corrected.
0b52def	Cancellation routing corrected.
a0f89d2	Delivery-instruction flow and punctuation corrected.
24147c3	Delivery-address flow added.
3de998a	Refund-status flow added.
4ee6124	Chat response area improved.
fac9d05	Payment-failure flow added; final clean checkpoint.

Appendix D - Glossary

Term	Definition
AI interpretation	Conversion of customer language into constrained structured intent.
Authoritative data	Trusted product records and state used for final decisions.
Deterministic rule	Predictable application logic that controls the outcome.
Fallback	Safe non-provider behaviour used when interpretation is unavailable.
Gemini	External AI provider used server-side for language interpretation.
Mock data	Fictional records used for safe prototyping and testing.
Mock provider	Local interpretation component used without a live external request.
RPD	Requests per day provider quota limit.
RTM	Requirements Traceability Matrix.
UAT	User Acceptance Testing against agreed business acceptance criteria.
Prototype closure	Formal acceptance of the agreed demonstration scope, not production release approval.

Conclusion

BiteBuddy reached formal prototype closure with a secure AI interpretation layer, deterministic business controls, resilient fallback, broad mobile-first customer support and a complete evidence trail. The project is suitable for a professional Business Analyst portfolio when presented with its mock-data and production boundaries clearly stated.

END OF DOCUMENT

BiteBuddy - Professional Project Delivery Documentation - Version 2.0